

Custom Visual Log Component for Lazarus / Free Pascal

Component: **AmpulseLog**
Version: 1.0.0
Base Class: TCustomListBox
Platform: Lazarus / Free Pascal / LCL

Overview

AmpulseLog is a visual logging component for Lazarus applications, designed to display application messages in an organized, colorful and easy-to-read format.

The component supports different log levels, timestamps, customizable font and background colors, automatic scrolling, memory limits, file operations and clipboard support.

Features

-  INFO — informative messages
-  SUCCESS / OK — success messages
-  WARNING / WARNING — warning messages
-  ERROR / ERROR — error messages
-  Date and time — timestamp of each message
-  Colored font — configurable colors per level
-  Colored background — configurable background colors per level
-  AutoScroll — automatic scrolling to the last record
-  Count — number of records currently stored
-  MaxItems — limits the number of records kept in memory
-  SaveToFile — saves the complete log content to a file
-  AppendToFile — adds new records to the file
-  Clipboard — copies log content to the clipboard
-  Clear — clears all logs
-  PopupMenu — supports LCL's TPopupMenu
-  Owner Draw — custom drawing of logs
-  Configurable Properties — colors and behavior can be changed via the Object Inspector

Log Levels

The component has four message levels:

Level	Prefix	Description
 Info	[INFO]	General information
 Success	[OK]	Operation successfully completed
 Warning	[WARNING]	Warning or situation requiring attention
 Error	[ERROR]	Error or failure in operation

Installation / Lazarus IDE

1. Download and extract the component folder to your computer.
2. Open **Lazarus IDE**.
3. In the top menu, go to Package -> Open package file (.lpk)...
4. Select the **ampulselog.lpk** file.
5. In the package window that opens, click the Use button and select **Install**.
6. Lazarus will ask if you want to rebuild the IDE. **Click Yes**.
7. The IDE will Rebuilding Lazarus automatically.
8. The TAmplitudeLog component will be available on the Common Controls palette tab.

Basic Usage

After placing an TAmplitudeLog on a form, messages can be added using the component's public methods.

Internally, the levels are represented by:

TLogLevel = (**llInfo**, **llSuccess**, **llWarning**, **llError**);

Custom Messages:

Adds an informational message.

```
AmpulseLog1.Info('Application started.');
```

Result : 15:53:35 [INFO] Application started.

Adds a success message.

```
AmpulseLog1.Success('Database connection established!');
```

Result : 15:53:35 [OK] Database connection established!

Adds a warning message.

```
AmpulseLog1.Warning('Configuration file was not found.');
```

Result : 15:53:35 [WARNING] Configuration file was not found.

Adds an error message.

```
AmpulseLog1.Error('Unable to connect to the server.');
```

Result : 15:53:35 [ERROR] Unable to connect to the server.

Adds a custom message to the log.

Example:

```
procedure Add(const AText: string; ACor: TColor);
AmpulseLog1.Add('Custom message!', clFuchsia);
```

Result : 15:53:35 [INFO] Custom message!

Control

Clear

Removes all log records.

```
procedure Clear;
```

Example:

```
AmpulseLog1.Clear;
```

Count

Returns the number of records currently stored.
function Count: Integer;

Example:
ShowMessage(
'Records: ' +
IntToStr(AmpulseLog1.Count));

File Operations :

SaveToFile

Saves the complete log content to a file.
procedure SaveToFile(const AFileName: string);

Example:
AmpulseLog1.SaveToFile(
'C:\MyApplication\Logs\Name.log');

+ AppendToFile

The component internally maintains record identifiers to control its file-recording process.
Appends log records to an existing file.
procedure AppendToFile(const AFileName: string);

Example:
AmpulseLog1.AppendToFile(
'C:\MyApplication\Logs\Name.log');

Clipboard

CopyToClipboard
Copies the log content to the system clipboard.
procedure CopyToClipboard;

Example:
AmpulseLog1.CopyToClipboard;

Font Colors

Each log level has its own configurable font color.

InfoColor

Default: clBlue
AmpulseLog1.InfoColor := clBlue;

SuccessColor

Default: clGreen
AmpulseLog1.SuccessColor := clGreen;

WarningColor

Default:clOlive
AmpulseLog1.WarningColor := clOlive;

ErrorColor

Default:clRed
AmpulseLog1.ErrorColor := clRed;

Background Colors

Each log level also has an independent background color.

InfoBackColor

Default:clWindow

SuccessBackColor

Default:clWindow

WarningBackColor

Default:clWindow

ErrorBackColor

Default:clWindow

Example:
AmpulseLog1.WarningBackColor := \$00DFFFFFF;
AmpulseLog1.ErrorBackColor := \$00DBFFFF;
AmpulseLog1.SuccessBackColor:= \$00E8FFE8;
AmpulseLog1.InfoBackColor:=\$00FAE7CD;

ShowTime

Controls whether the date and time are displayed. Default:True

```
AmpulseLog1.ShowTime := False;
AmpulseLog1.Add( 'Custom message!',clFuchsia);
Result : [INFO] Custom message!
```

```
AmpulseLog1.ShowTime := True;
AmpulseLog1.Add( 'Custom message!',clFuchsia);
Result : 15:53:35 [INFO] Custom message!
```

TimeFormat

Defines the format used for timestamps.

 **Time only:**

```
AmpulseLog1.TimeFormat := 'hh:nn:ss';
Result : 15:53:35 Custom message!
```

 **Date and time:**

```
AmpulseLog1.TimeFormat := 'dd/mm/yyyy hh:nn:ss';
Result : 23/08/26 15:53:35 Custom message!
```

ShowLevel

Controls whether the log level prefix is displayed. Default:True

Examples:

```
AmpulseLog1.ShowLevel := False;
AmpulseLog1.Add( 'Custom message!',clFuchsia);
Result : 15:53:35 Custom message!
```

```
AmpulseLog1.ShowLevel := True;
AmpulseLog1.Add( 'Custom message!',clFuchsia);
Result : 15:53:35 [INFO] Custom message!
```

```
Example: ShowTime = False and ShowLevel = False
AmpulseLog1.Add( 'Custom message!',clFuchsia);
Result : Custom message!
```

Behavior

 AutoScroll

Controls automatic scrolling to the newest record.
AmpulseLog1.AutoScroll := True;
This feature is especially useful for real-time application logs.

 MaxItems

Defines the maximum number of records retained in memory.
Example: AmpulseLog1.MaxItems := 1000;
A value of 0 means that no explicit maximum limit is configured.
Default :0

Internal Data Structure

Each log record is internally represented by TLogItem.

TLogItem = record

ID: Int64;	Field	Description / Descrição
DataHora: TdateTime;	-----	-----
Texto: string;	ID	Sequential record identifier
Cor: Tcolor;	DataHora	Date and time
Nivel: TlogLevel;	Texto	Message text
end;	Cor	Font color
	Nivel	Log level

Owner Draw

TAmplitudeLog uses owner-draw rendering to provide customized visual presentation for each log record.

This allows the component to independently control:

timestamp color;
level color;
message color;
background color;
level prefix;
message position.

The custom rendering is implemented through:

```
procedure DrawItem(  
  Index: Integer;  
  ARect: TRect;  
  State: TOwnerDrawState  
); override;
```

PopupMenu

TAmplitudeLog supports the standard LCL TPopupMenu.

A popup menu can be assigned through the Object Inspector or programmatically.

Example:

```
AmpulseLog1.PopupMenu := PopupMenu1;
```

Properties

The following properties are available through the Lazarus Object Inspector.

Font Colors

Property	Default
-----	-----
InfoColor	clBlue
SuccessColor	clGreen
WarningColor	clOlive
ErrorColor	clRed

Background Colors

Property	Default
-----	-----
InfoBackColor	clWindow
SuccessBackColor	clWindow
WarningBackColor	clWindow
ErrorBackColor	clWindow

Behavior

Property	Default
-----	-----
AutoScroll	True
MaxItems	0
TimeFormat	hh:nn:ss

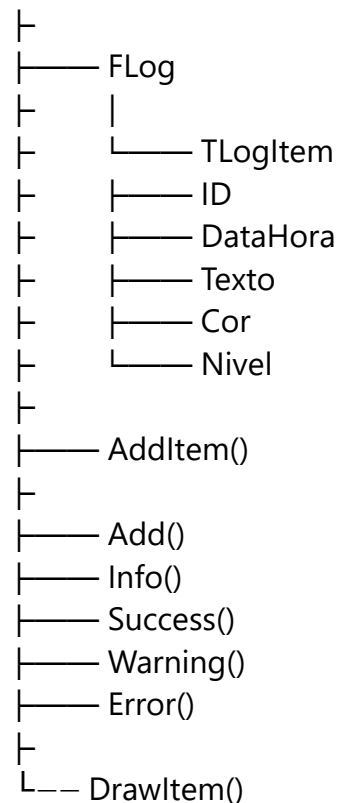
Display

Property	Default
-----	-----
ShowTime	True
ShowLevel	True

Architecture

The internal architecture can be summarized as follows:

TAmplitudeLog



FLog is the logical source of the stored log records.

The TCustomListBox functionality provides the visual infrastructure required for displaying, scrolling and owner-drawing the records.

Recommended API

The recommended way to add records is through the methods provided specifically by TAmplitudeLog: Use the TAmplitudeLog methods to add messages. This keeps the internal log data synchronized and ensures that timestamps, levels, colors and other internal information are correctly maintained.

ampulseLog is a simple and easy-to-learn component.

Developed with clear and complete documentation of its functionalities, its structure facilitates understanding, allowing any developer with basic knowledge to easily make changes and implement new features.

The AmpulseLog was originally developed as part of the AmpulseServer application's logging system, with the goal of offering a simple solution for recording and tracking application events.

Later, the component was separated and is being made available independently as a courtesy to the Lazarus/Free Pascal community, allowing other developers to use it in their own projects.

<https://github.com/AncelmoSilva/ampulselog/>

Copyright (C) 2026 R. Ancelmo Silva

Contacts: suporte@raquel.adm.br